Computación de Alto Rendimiento

Grado en Ingeniería Informática

PRÁCTICA DE AULA

Práctica 3 · Tema 3

Bucles, datos y tareas con OpenMP

Práctica · Núcleo de 2 h en clase y trabajo autónomo · Grupos de 4

Docentes: Elam Uceda Herrero · Javier Vázquez Pereda



Ficha de la práctica

Esta es la práctica del Tema 3, y recorre el tema entero: cómo se reparten las iteraciones de un bucle, qué pasa con los datos dentro de la región paralela y cuándo compensa partir el trabajo en tareas. Se hace **al terminar la teoría del tema**. En el Tema 2 escribiste los hilos a mano: crearlos, repartir los índices, esperar y recoger. Aquí eso se escribe en una línea, y el trabajo pasa a ser otro: **decidir qué se puede paralelizar, cómo se reparte y qué comparten los hilos sin que lo hayas pedido**. Los programas de hoy, y sobre todo el método de medida, son la base del Laboratorio 1, TermoHPC I.

APARTADO	DETALLE
Asignatura	Computación de Alto Rendimiento · 4.º de Ingeniería Informática · Tema 3, *Programación con OpenMP*
Organización	**Núcleo en clase**: una sesión de 2 horas, ejercicios 1, 2 y 3. **Trabajo autónomo obligatorio**: unas 3 horas fuera del aula, ejercicios 4 y 5. Los cinco ejercicios puntúan
Modalidad	Grupos de 4 personas. Un solo juego de programas por grupo, y todos los nombres en la entrega
Entorno	`gcc` con `-fopenmp`, compilando con `-O2` y, en el ejercicio 4, también con `-O0`. Sirve Windows (el gcc de CLion o de MinGW), Linux o WSL
Requisito previo	Haber estudiado los apartados **3.1 a 3.4**, tener el `crono.h` de la Práctica 2 y llegar con el entorno comprobado: `comprueba_omp.c` tiene que decir **ENTORNO LISTO** antes de la sesión
Qué se entrega	**Un único PDF por equipo**, con el código pegado dentro, las tablas de medidas, las gráficas que se piden, las conclusiones y la declaración del trabajo y del uso de IA. No se entregan ficheros de código sueltos ni comprimidos
Peso	Los 5 ejercicios suman **10 puntos**: 6 el núcleo en clase y 4 el trabajo autónomo

CÓMO REPARTIR LAS DOS HORAS DE CLASE

Es una estimación, no una norma. Si vas muy por detrás de este reparto, avisa en clase.

Leer el guion, 10 min · Ejercicio 1, 25 min · Ejercicio 2, 35 min · Ejercicio 3, 35 min · conclusiones y dudas, 15 min. Lo que no dé tiempo a redactar en clase se completa en casa, junto con el trabajo autónomo.

Objetivo

OpenMP es engañosamente fácil: se pone una línea encima de un bucle y el programa va más rápido. La mitad de las veces. Esta práctica trata de la otra mitad, y no trae teoría nueva: todo está en los apartados 3.1 a 3.4, y cada ejercicio dice de qué apartado sale. Al terminar tienes que poder responder, con números propios:

- ¿Cómo se reconoce antes de ejecutar que un bucle no se puede paralelizar tal cual, y qué pinta tiene el fallo cuando se escapa?
- ¿Por qué repartir las iteraciones por igual no es repartir el trabajo, y qué planificación lo arregla?
- ¿Cuánto cuesta cada forma de acumular un resultado — `critical`, `atomic` y `reduction` —, y por qué la comparación contradice a muchos manuales?
- ¿Qué comparten los hilos sin que lo hayas pedido, una variable o una línea de caché, y cómo se nota cada una?
- ¿Cuánto trabajo tiene que tener una tarea para que compense crearla?

Antes de la sesión: el entorno, comprobado

La sesión de clase no tiene tiempo para instalar nada. **Llega con esto hecho.** Si el comprobador no dice ENTORNO LISTO, avisa al profesor antes del día de la práctica, no durante.

Copia en tu carpeta el `crono.h` de la Práctica 2 y compila este comprobador con `-fopenmp`:

```c
/* comprueba_omp.c - ejecutalo ANTES de la sesion. Si no dice ENTORNO LISTO,
   la practica no se puede hacer en clase. */
#include <stdio.h>
#include <omp.h>
#include "crono.h"

int main(void)
{
❶ #ifndef _OPENMP
    printf("ALGO FALLA: se ha compilado sin -fopenmp\n");
    return 1;
#else
    int en_region = 0;
❷   #pragma omp parallel reduction(+:en_region)
    en_region += 1;                     /* cada hilo suma uno a su copia */

    double paso = 1.0;                  /* el menor salto que da el reloj */
    for (int i = 0; i < 1000; i++) {
        double a = reloj(), b;
❸       do b = reloj(); while (b == a);
        if (b - a < paso) paso = b - a;
    }
    int maximo = omp_get_max_threads();
    printf("procesadores logicos (omp_get_num_procs)  : %d\n", omp_get_num_procs());
    printf("hilos por defecto (omp_get_max_threads)   : %d\n", maximo);
    printf("hilos que han ejecutado la region         : %d\n", en_region);
    printf("resolucion del reloj de crono.h           : %.0f ns\n", paso * 1e9);
    printf("%s\n", (en_region > 1 && en_region == maximo && paso < 1e-6)
                   ? "ENTORNO LISTO" : "ALGO FALLA: avisa antes de la sesion");
    return 0;
#endif
}
```

❶ Si se compila sin `-fopenmp`, la macro `_OPENMP` no existe y el programa lo dice. Es el fallo del apartado 3.1.4: sin el flag, las directivas se ignoran en silencio y no hay ningún error. ❷ Cada hilo suma uno a su propia copia y al final se juntan, así que `en_region` vale el número de hilos que han ejecutado la región. Es la cláusula `reduction`: hoy solo la usas ya escrita, y se estudia en el apartado 3.3.6. ❸ El salto mínimo del reloj, como en la Práctica 2.

```
$ gcc -O2 -fopenmp -Wall -Wextra -o comprueba_omp comprueba_omp.c
$ ./comprueba_omp
procesadores logicos (omp_get_num_procs)  : 24
hilos por defecto (omp_get_max_threads)   : 24
hilos que han ejecutado la region         : 24
resolucion del reloj de crono.h           : 100 ns
ENTORNO LISTO
```

Esa es la salida en Windows 11. En el mismo portátil con Linux (WSL 2, Ubuntu 24.04) el reloj salta de 19 ns. Y si compilas sin `-fopenmp`, el programa responde «ALGO FALLA: se ha compilado sin -fopenmp»: compruébalo una vez a propósito.

Anota también cuántos núcleos físicos tiene tu máquina, porque OpenMP solo te dice los procesadores lógicos y vas a necesitar el dato para leer los speedups. En Linux o WSL, la orden `lscpu` da las líneas *Core(s) per socket* y *Thread(s) per core*. En Windows, en PowerShell: `Get-CimInstance Win32_Processor | Select-Object NumberOfCores, NumberOfLogicalProcessors`.

Prepárate también para compilar el mismo fichero con `-O0`, que hace falta en el ejercicio 4: es la misma orden cambiando `-O2` por `-O0`, y conviene haberlo probado una vez antes.

Antes de empezar

La chuleta

NECESITAS	SE ESCRIBE ASÍ	APARTADO
Abrir una región paralela	`#pragma omp parallel`	3.1.3
Fijar los hilos	`num_threads(8)` · `omp_set_num_threads(8)` · `OMP_NUM_THREADS`	3.1.6
Repartir un bucle	`#pragma omp parallel for`	3.2.1 y 3.2.2
Elegir el reparto	`schedule(static)` · `dynamic` · `guided`	3.2.4
Cambiarlo sin recompilar	`schedule(runtime)` + `omp_set_schedule(omp_sched_dynamic, 64)`	3.2.4 y 3.2.5
Variable propia de cada hilo	`private(t)`, o declararla dentro del bucle	3.3.2 y 3.3.3
Obligarte a declararlo todo	`default(none)` `shared(x, y)`	3.3.5
Acumular bien y barato	`reduction(+:s)`	3.3.6
Sección crítica y actualización atómica	`#pragma omp critical` · `#pragma omp atomic`	3.3.7 y 3.3.8
Evitar la falsa compartición	separar las casillas de cada hilo una línea de caché entera	3.3.12
Crear tareas y esperarlas	`#pragma omp task` · `#pragma omp taskwait`, dentro de `parallel` + `single`	3.4.3
Cortar la recursión	por debajo de un umbral, la versión secuencial	3.4.5
Reloj fino	`reloj()` de `crono.h`	Práctica 2
Ver el ensamblador	`gcc -O2 -fopenmp -S -o salida.s programa.c`	—

Las trampas de hoy

Trampa 1: el reloj de OpenMP es grueso en Windows

`omp_get_wtime()` parece el reloj natural, pero con MinGW en Windows salta de milisegundo en milisegundo. Muchos tiempos de esta práctica están entre 1 y 20 ms: medirlos con él es medirlos con un error de entre el 5 % y el 100 %. Usa `crono.h`.

Trampa 2: -O2 borra lo que no se usa, y la referencia secuencial tiene que serlo de verdad

Si el bucle calcula algo que nadie lee, el compilador lo elimina y mides cero: por eso todos los programas consumen el resultado al final. Y la referencia secuencial tiene que ser un bucle **sin directiva**. Un bucle con la directiva y `if(0)`, o con un solo hilo, sigue pasando por el planificador de OpenMP: en la preparación de esta práctica, una iteración que sin directiva costaba 0,95 ns costaba así 23 ns, y los speedups salían de 50 sin ser verdad.

Trampa 3: un programa mal paralelizado puede dar bien

Un bucle con dependencias puede fallar solo en unas pocas posiciones de un millón. Y un temporal compartido puede dar el resultado correcto en todas las ejecuciones con `-O2` y fallar en todas con `-O0`, **con el mismo fichero fuente**: es el corazón del ejercicio 4. Compara siempre con la versión secuencial posición a posición, repite, y antes de concluir nada sobre corrección compila las dos versiones. Que salga bien una vez no demuestra nada.

Trampa 4: el volatile que hace falta para ver el efecto

Si una variable se queda en un registro del procesador, no hay memoria compartida de por medio y los fenómenos del ejercicio 4 no aparecen. `volatile` obliga a la variable a vivir en memoria. Allí no es un arreglo: es lo que hace visible lo que se estudia.

Trampa 5: en coma flotante, el orden de la suma cambia el resultado

Sumar los mismos números en otro orden puede cambiar los últimos decimales, y un `reduction` suma en un orden distinto del bucle secuencial. Por eso los programas de hoy suman valores enteros guardados en `double`: así la suma es exacta en cualquier orden y se puede comparar con `==`. Con decimales, se compara con tolerancia.

Trampa 6: no estás solo en la máquina

Repite cada medida varias veces y quédate con el mínimo, y di cuántas repeticiones hay detrás de cada número.

ANTES DE JUZGAR NINGÚN SPEEDUP La máquina de referencia tiene 12 núcleos físicos de dos clases, unos rápidos y otros compactos, y 24 procesadores lógicos. El speedup lineal no es alcanzable, y no por culpa de tu programa. Cuando midas 6 con 24 hilos, no escribas «el escalado es pobre»: explica de dónde sale el techo. Se evalúa la explicación, no el número.

> **SOBRE LOS NÚMEROS DE ESTE GUION** Todas las tablas y gráficas se midieron en septiembre de 2026 con los programas de este guion —las del ejercicio 5, el día 30—, en un AMD Ryzen AI 9 HX 370 con Windows 11 y gcc 14.2, y se repitieron en el mismo portátil con WSL 2 (Ubuntu 24.04) y gcc 13.3. Todos compilan sin avisos con `-O2 -fopenmp -Wall -Wextra`. Tus cifras serán distintas; lo que se corrige es la forma y la explicación.

Núcleo en clase: ejercicios 1 a 3 (6 puntos)

Los programas se dan **casi completos**: en cada uno falta una pieza que escribes tú, y es justo la que enseña el ejercicio. Compila siempre con `-O2 -fopenmp -Wall -Wextra` y `-lm` si usas funciones matemáticas.

Ejercicio 1 (1,5 puntos) — Cuáles se pueden y cuáles no

Enunciado. `p3_ej1.c` ejecuta cinco bucles, cada uno primero en serie —esa es la respuesta correcta— y después diez veces con `#pragma omp parallel for` puesto sin pensar, y cuenta en cuántas ejecuciones falla y cuántas posiciones salen mal (apartado 3.2.3).

- **Antes de ejecutar**, clasifica por escrito los bucles 1 a 4 con la tabla del apartado 3.2.3 —sin dependencias, dependencia de flujo, antidependencia o acumulador compartido— y di cuáles van a fallar. El bucle 5 es el 4 con la cláusula `reduction` del apartado 3.3.6: está ahí para que veas el arreglo funcionando, y en el ejercicio 3 medirás lo que cuesta cada forma de acumular.
- **Tu parte**: el bucle 3, `b[i] = b[i+1] + 1`, se puede paralelizar si cada iteración lee el valor que había **antes** de empezar el bucle. Escríbelo en el sitio marcado y comprueba que ya no falla ninguna vez.

```c
/* p3_ej1.c - cuales se pueden paralelizar y cuales no (apartado 3.2.3).
   Cada bucle se ejecuta primero en serie (la respuesta correcta) y despues
   EJEC veces con la directiva puesta; se cuenta cuantas veces falla. */
#include <stdio.h>
#include <string.h>
#include <math.h>
#include <omp.h>

#define N    2000000
#define EJEC 10

static double a[N], b[N], ref[N], copia[N];

static void inicial(double *v)
{
    for (int i = 0; i < N; i++) v[i] = (double)(i % 7) + 1.0;
}

/* Posiciones en las que v no coincide con la referencia. */
static long long difiere(const double *v)
{
    long long d = 0;
    for (int i = 0; i < N; i++)
        if (v[i] != ref[i]) d++;
    return d;
}

static void informa(const char *nombre, int malas, long long peor)
{
    printf("%-34s %4d/%d %14lld\n", nombre, malas, EJEC, peor);
}
```

```c
int main(void)
{
    printf("%d hilos, %d posiciones, %d ejecuciones por bucle\n",
           omp_get_max_threads(), N, EJEC);
    printf("bucle                              fallos   posiciones mal (peor)\n");

    /* 1) sin dependencias: sirve de control */
    inicial(ref);
    for (int i = 0; i < N; i++) ref[i] = 2.0 * ref[i];
    {
        int malas = 0; long long peor = 0;
        for (int e = 0; e < EJEC; e++) {
            inicial(b);
            #pragma omp parallel for
            for (int i = 0; i < N; i++) b[i] = 2.0 * b[i];
            long long d = difiere(b);
            if (d) { malas++; if (d > peor) peor = d; }
        }
        informa("1 b[i] = 2*b[i]", malas, peor);
    }

    /* 2) dependencia de flujo: cada vuelta necesita la anterior */
    inicial(a);
    ref[0] = a[0];
    for (int i = 1; i < N; i++) ref[i] = ref[i - 1] + a[i];
    {
        int malas = 0; long long peor = 0;
        for (int e = 0; e < EJEC; e++) {
            inicial(b);
            #pragma omp parallel for
❶           for (int i = 1; i < N; i++) b[i] = b[i - 1] + b[i];
            long long d = difiere(b);
            if (d) { malas++; if (d > peor) peor = d; }
        }
        informa("2 b[i] = b[i-1] + b[i]", malas, peor);
    }

    /* 3) antidependencia: se lee una posicion que otro hilo va a escribir */
    inicial(ref);
    for (int i = 0; i < N - 1; i++) ref[i] = ref[i + 1] + 1.0;
    {
        int malas = 0; long long peor = 0;
        for (int e = 0; e < EJEC; e++) {
            inicial(b);
            #pragma omp parallel for
❷           for (int i = 0; i < N - 1; i++) b[i] = b[i + 1] + 1.0;
            long long d = difiere(b);
            if (d) { malas++; if (d > peor) peor = d; }
        }
        informa("3 b[i] = b[i+1] + 1", malas, peor);
    }

❸   /* 3 arreglado. EJERCICIO 1: escribe aqui el bucle 3 en paralelo y sin
       fallos. Repitelo EJEC veces como los anteriores, cuenta los fallos
       y llama a informa("3 arreglado", malas, peor). */

❹   /* 4) y 5) un acumulador compartido, sin y con reduction */
```

```
    inicial(a);
    double s_ref = 0.0;
    for (int i = 0; i < N; i++) s_ref += a[i];
    {
        int malas = 0; double peor = 0.0;
        for (int e = 0; e < EJEC; e++) {
            double s = 0.0;
            #pragma omp parallel for
            for (int i = 0; i < N; i++) s += a[i];
            double err = fabs(s - s_ref) / s_ref;
            if (err > 0.0) { malas++; if (err > peor) peor = err; }
        }
        printf("%-34s %4d/%d   error relativo %.3f\n",
               "4 s += a[i] (sin reduction)", malas, EJEC, peor);
    }
    {
        int malas = 0;
        for (int e = 0; e < EJEC; e++) {
            double s = 0.0;
            #pragma omp parallel for reduction(+:s)
            for (int i = 0; i < N; i++) s += a[i];
            if (s != s_ref) malas++;
        }
        printf("%-34s %4d/%d\n", "5 s += a[i] (con reduction)", malas, EJEC);
    }
    return 0;
}
```

❶ La dependencia de flujo: cada elemento necesita el valor ya calculado del anterior. ❷ La antidependencia: se lee `b[i+1]`, que el hilo del trozo siguiente puede haber sobrescrito ya. ❸ Aquí va tu versión. ❹ El acumulador compartido, primero sin `reduction` y después con ella.

Resultado con tu bucle 3 ya escrito, en la máquina de referencia con 24 hilos:

```
bucle                            fallos   posiciones mal (peor)
1 b[i] = 2*b[i]                    0/10              0
2 b[i] = b[i-1] + b[i]            10/10        1916665
3 b[i] = b[i+1] + 1               10/10             22
3 arreglado                        0/10              0
4 s += a[i] (sin reduction)       10/10   error relativo 0.958
5 s += a[i] (con reduction)        0/10
```

Mira la tercera fila. La antidependencia falla las diez veces, pero **solo en 22 posiciones de dos millones**, un 0,001 %. Y no están repartidas al azar: están en las fronteras entre los trozos de los hilos, y con 24 hilos hay 23 fronteras. Es el fallo más peligroso de los cinco precisamente porque casi no se ve: si tu prueba compara solo la suma total, o unas cuantas posiciones al azar, te dirá que está bien. En Linux sale igual: 22 posiciones y un error relativo de 0,917 en el acumulador.

> **PISTA** La idea es la de la teoría: si el valor que lee cada iteración no depende de lo que escriban las demás, las iteraciones son independientes. Una copia del array hecha antes del bucle, con `memcpy` (ya está incluida `string.h` y declarado el array `copia`), lo consigue. Comprueba que tu versión da 0 fallos en las diez ejecuciones.

Qué va en el PDF

- Tu clasificación previa de los cinco bucles, escrita antes de ejecutar, y la comparación con lo que salió.
- Tu bucle 3 arreglado, pegado como texto, y la tabla completa de fallos.
- Dos o tres conclusiones justificadas: por qué el bucle 3 falla en tan pocas posiciones, con el número de fronteras de tu máquina, y por qué tu arreglo no tiene ese problema.

Ejercicio 2 (2,0 puntos) — Un bucle desequilibrado y las planificaciones

Enunciado. `p3_ej2.c` ejecuta un bucle triangular, en el que la iteración *i* cuesta proporcional a *i*, con cinco planificaciones del apartado 3.2.4, y compara sus tiempos con el secuencial. Los tiempos solos no explican nada. **Tu parte** es la función `reparto`, que cuenta cuánto trabajo se lleva cada hilo con cada planificación (apartado 3.2.6).

```c
/* p3_ej2.c - un bucle desequilibrado y las planificaciones (apartado 3.2.4).
   La iteracion i cuesta proporcional a i. */
#include <stdio.h>
#include <omp.h>
#include "crono.h"

#define N    20000
#define REP  5
#define MAXH 64

static double r[N];

/* Trabajo de la iteracion i: i sumas en coma flotante. */
static double trabajo(int i)
{
    double s = 0.0;
❶   for (int k = 0; k < i; k++) s += (double)k * 1e-9;
    return s;
}

static double secuencial(void)
{
    double t0 = reloj();
    for (int i = 0; i < N; i++) r[i] = trabajo(i);
    return reloj() - t0;
}

/* Mejor de REP con la planificacion elegida en tiempo de ejecucion. */
static double paralelo(omp_sched_t tipo, int bloque)
{
❷   omp_set_schedule(tipo, bloque);
    double mejor = 1e30;
    for (int rep = 0; rep < REP; rep++) {
        double t0 = reloj();
❸       #pragma omp parallel for schedule(runtime)
        for (int i = 0; i < N; i++) r[i] = trabajo(i);
        double t = reloj() - t0;
        if (t < mejor) mejor = t;
    }
    return mejor;
}
```

```
/* EJERCICIO 2: cuanto trabajo se lleva cada hilo con esa planificacion.
   Ejecuta el bucle real con schedule(runtime), suma el coste de cada
   iteracion (vale i) en la casilla del hilo que la ejecuta, imprime cuantas
   veces la media se lleva el mas cargado y cuantos hilos no han trabajado,
   y devuelve ese factor. */
❹ static double reparto(omp_sched_t tipo, int bloque)
{
    /* tu codigo */
}

int main(void)
{
    struct { const char *nombre; omp_sched_t tipo; int bloque; } p[] = {
        {"static (por defecto)", omp_sched_static, 0},
        {"static, 1",            omp_sched_static, 1},
        {"dynamic (bloque 1)",   omp_sched_dynamic, 1},
        {"dynamic, 64",          omp_sched_dynamic, 64},
        {"guided",               omp_sched_guided, 0},
    };
    printf("%d hilos, %d iteraciones, mejor de %d\n", omp_get_max_threads(), N, REP);
    double ts = 1e30;
    for (int rep = 0; rep < REP; rep++) { double t = secuencial(); if (t < ts) ts = t; }
    double control = 0.0;
    for (int i = 0; i < N; i++) control += r[i];
    printf("%-22s %8.4f s  %5.2fx\n", "secuencial", ts, 1.0);
    for (int k = 0; k < 5; k++) {
        double t = paralelo(p[k].tipo, p[k].bloque);
        printf("%-22s %8.4f s  %5.2fx\n", p[k].nombre, t, ts / t);
    }
    double comprueba = 0.0;
    for (int i = 0; i < N; i++) comprueba += r[i];
    printf("resultado %s\n", comprueba == control ? "identico al secuencial" : "DISTINTO");
    printf("reparto del trabajo entre los %d hilos\n", omp_get_max_threads());
    for (int k = 0; k < 5; k++) {
        printf("%-22s", p[k].nombre);
        reparto(p[k].tipo, p[k].bloque);
    }
    return 0;
}
```

❶ El coste de la iteración *i* son *i* sumas: la última cuesta veinte mil veces más que la primera. ❷ Antes de cada medida se fija la planificación con `omp_set_schedule`. ❸ Con `schedule(runtime)` la planificación se elige al ejecutar, así que el mismo bucle sirve para todas (apartado 3.2.4). ❹ Aquí va tu función.

Resultado en la máquina de referencia con Windows y 24 hilos, mejor de 5:

```
secuencial               0.1510 s   1.00x
static (por defecto)     0.0179 s   8.42x
static, 1                0.0141 s  10.75x
dynamic (bloque 1)       0.0116 s  13.05x
dynamic, 64              0.0122 s  12.34x
guided                   0.0128 s  11.82x
resultado identico al secuencial
reparto del trabajo entre los 24 hilos
static (por defecto)    mas cargado  1.96 veces la media   hilos sin trabajo  0
static, 1               mas cargado  1.00 veces la media   hilos sin trabajo  0
dynamic (bloque 1)      mas cargado  1.38 veces la media   hilos sin trabajo  0
dynamic, 64             mas cargado  1.46 veces la media   hilos sin trabajo  0
guided                  mas cargado  1.60 veces la media   hilos sin trabajo  0
```

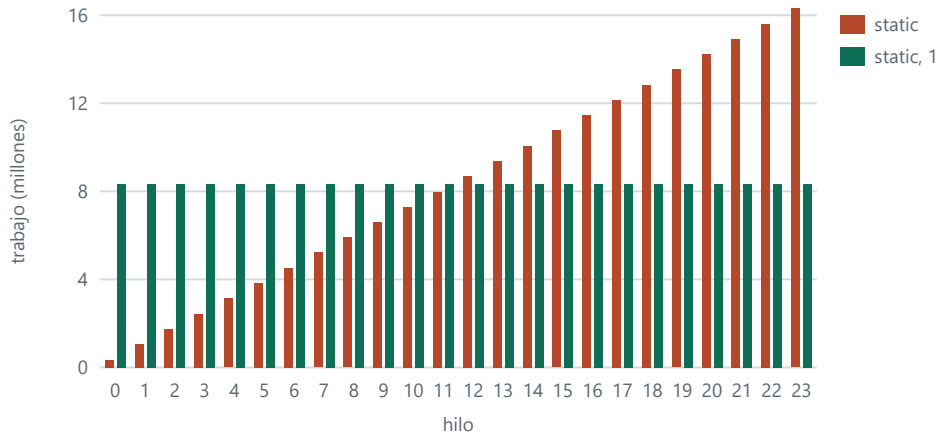


Figura P3.1 · *Ejercicio 2. Trabajo de cada hilo en el bucle triangular. Con static todos reciben prácticamente las mismas iteraciones, pero el último hace 47 veces más trabajo que el primero. Con static, 1 cada hilo recibe iteraciones caras y baratas en la misma proporción y las 24 barras quedan iguales. Medido con el mismo bucle en la máquina de referencia con Windows.*

Con `static` cada hilo recibe unas 833 iteraciones, y sin embargo el hilo que se lleva el final del índice trabaja casi el doble de la media y 47 veces más que el primero. **Repartir iteraciones no es repartir trabajo**. Y como al final del bucle hay una barrera implícita, los hilos que acaban pronto esperan al más cargado: por eso `static` es aquí la más lenta (apartado 3.2.6).

`static, 1` reparte exactamente —cada hilo recibe iteraciones caras y baratas en la misma proporción— y aun así pierde contra `dynamic`. En Linux, en la misma máquina, salió incluso más lenta que `static`: 6,2 frente a 7,2. Un reparto perfecto sobre el papel no es necesariamente el más rápido, y por eso hay que medir. Y ojo con `dynamic`: reparte a quien pide, así que su reparto cambia con el sistema y con la ejecución. En Windows, tres ejecuciones dejaron al más cargado en 1,38, 1,41 y 1,30 veces la media; en Linux, en 1,94, casi como `static`, y aun así fue la más rápida.

> **PISTA** Lo que necesitas es un array `long long carga[MAXH] = {0}`, local a la función, y dentro del bucle real —con `r[i] = trabajo(i)`— sumar `i` en `carga[omp_get_thread_num()]`. Cada hilo escribe solo en su casilla, así que no hay carrera. Cuéntalo sobre el bucle con el trabajo de verdad: si cuentas en un bucle vacío, un solo hilo lo hace casi todo antes de que los demás arranquen y el reparto no se parece al real.

Qué va en el PDF

- Tu función `reparto`, pegada como texto.
- La tabla de tiempos de las cinco planificaciones y la de reparto.
- Una gráfica de barras con el trabajo de cada hilo, al menos para `static` y para `dynamic`.
- Dos o tres conclusiones justificadas: por qué `static` es la más lenta aunque reparta iteraciones por igual, y por qué `static, 1` reparte exacto y no gana.

Ejercicio 3 (2,5 puntos) — La comparación que contradice al manual

Enunciado. `p3_ej3.c` hace veinte millones de sumas con 8 hilos de cuatro formas —secuencial, `critical`, `atomic` y `reduction`— con un acumulador `double` (apartados 3.3.6 a 3.3.9). Muchos manuales dicen que `atomic` es más barato que `critical`. Vas a comprobarlo y, si la medida dice otra cosa, a explicar por qué.

- **Tu parte**: las mismas cuatro funciones con un acumulador `long long`. Es el **experimento de control**: si la explicación depende del tipo, al cambiar el tipo tiene que cambiar el resultado.
- Después, genera el ensamblador con `gcc -O2 -fopenmp -S` y busca qué instrucción ha puesto el compilador en cada `atomic`: cuenta las líneas con `lock cmpxchg` y con `lock add`.

```c
/* p3_ej3.c - critical, atomic y reduction sobre el mismo acumulado, con un
   acumulador double y con uno long long (apartados 3.3.6 a 3.3.9). */
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#include "crono.h"

#define N   20000000
#define REP 3
#define P   8               /* hilos de todas las medidas paralelas */

static double *x;           /* x[i] y v[i] valen lo mismo: (i % 7) + 1 */
static long long *v;

/* ---------- acumulador double ---------- */

static double d_secuencial(double *r)
{
    double s = 0.0, t0 = reloj();
    for (int i = 0; i < N; i++)
        s += x[i];
    *r = s;
    return reloj() - t0;
}

static double d_critical(double *r)
{
    double s = 0.0, t0 = reloj();
    #pragma omp parallel for num_threads(P)
    for (int i = 0; i < N; i++)
        #pragma omp critical
        s += x[i];
    *r = s;
    return reloj() - t0;
}
```

```
static double d_atomic(double *r)
{
    double s = 0.0, t0 = reloj();
    #pragma omp parallel for num_threads(P)
    for (int i = 0; i < N; i++)
❷       #pragma omp atomic
        s += x[i];
    *r = s;
    return reloj() - t0;
}

static double d_reduction(double *r)
{
    double s = 0.0, t0 = reloj();
❸   #pragma omp parallel for num_threads(P) reduction(+:s)
    for (int i = 0; i < N; i++)
        s += x[i];
    *r = s;
    return reloj() - t0;
}

/* ---------- acumulador long long ----------
❹   EJERCICIO 3: escribe l_secuencial, l_critical, l_atomic y l_reduction: las
   cuatro de arriba, pero con un acumulador long long que suma v[i]. Devuelven
   el tiempo y dejan en *r el resultado convertido a double. */

/* Mejor de REP; en *r deja el resultado del acumulado. */
static double mejor(double (*f)(double *), double *r)
{
    double m = 1e30;
    for (int k = 0; k < REP; k++) {
        double t = f(r);
        if (t < m) m = t;
    }
    return m;
}

static int bien = 1;

static void tanda(const char *tipo, double (*sec)(double *), double (*cri)(double *),
                  double (*ato)(double *), double (*red)(double *))
{
    double rs, rc, ra, rr;
    double ts = mejor(sec, &rs), tc = mejor(cri, &rc);
    double ta = mejor(ato, &ra), tr = mejor(red, &rr);
    printf("acumulador de tipo %s\n", tipo);
    printf("  secuencial  %8.4f s\n", ts);
    printf("  critical    %8.4f s\n", tc);
    printf("  atomic      %8.4f s   %6.2f veces mas rapido que critical\n", ta, tc / ta);
    printf("  reduction   %8.4f s   %6.0f veces mas rapido que critical\n", tr, tc / tr);
    if (rc != rs || ra != rs || rr != rs) bien = 0;
}

int main(void)
{
    x = malloc(N * sizeof *x);
    v = malloc(N * sizeof *v);
```

```
    if (!x || !v) { fprintf(stderr, "sin memoria\n"); return 1; }
    for (int i = 0; i < N; i++) { v[i] = i % 7 + 1; x[i] = (double)v[i]; }
    printf("%d hilos, %d sumas, mejor de %d\n", P, N, REP);
    tanda("double", d_secuencial, d_critical, d_atomic, d_reduction);
    tanda("long long", l_secuencial, l_critical, l_atomic, l_reduction);
    printf("los ocho acumulados dan %s\n", bien ? "lo mismo" : "DISTINTO");
    free(x);
    free(v);
    return 0;
}
```

❶ Con `critical`, la suma la hace un hilo cada vez y el turno lo gestiona un cerrojo. ❷ Con `atomic`, se pide que la actualización sea indivisible sin cerrojo. ❸ Con `reduction`, cada hilo acumula en su copia y se combinan al final: es el acumulador privado del ejercicio 3 de la Práctica 2, escrito en una cláusula. ❹ Aquí van tus cuatro funciones.

Resultado en la máquina de referencia con Windows, y el ensamblador:

```
8 hilos, 20000000 sumas, mejor de 3
acumulador de tipo double
  secuencial    0.0118 s
  critical      0.7234 s
  atomic        0.7938 s     0.91 veces mas rapido que critical
  reduction     0.0032 s      228 veces mas rapido que critical
acumulador de tipo long long
  secuencial    0.0062 s
  critical      0.6908 s
  atomic        0.2166 s     3.19 veces mas rapido que critical
  reduction     0.0028 s      247 veces mas rapido que critical
los ocho acumulados dan lo mismo

$ gcc -O2 -fopenmp -S -o p3_ej3.s p3_ej3.c
$ grep -c 'lock cmpxchg' p3_ej3.s
2
$ grep -c 'lock add' p3_ej3.s
2
```

Con `double`, `atomic` tarda 0,7938 s y `critical` 0,7234 s: no solo no mejora, es algo más lento. Con `long long`, `atomic` es **3,19 veces** más rápido. Las mismas directivas, el mismo número de sumas, y un comportamiento que depende **solo del tipo del acumulador**.

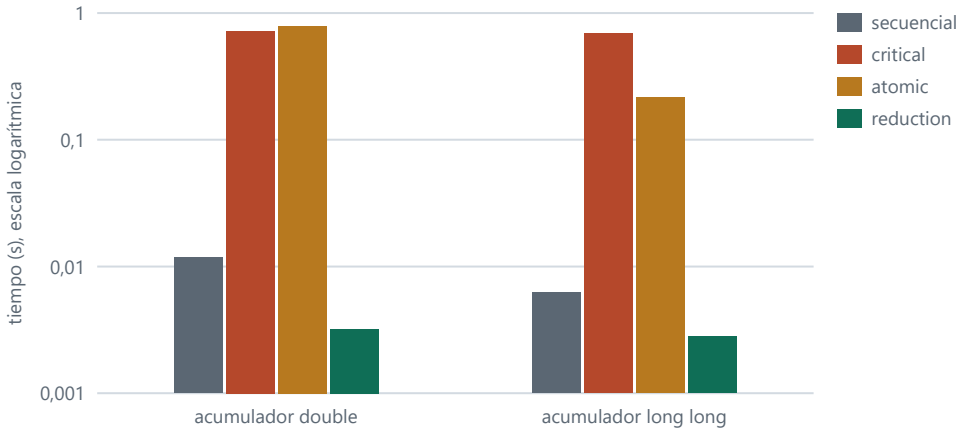


Figura P3.2 · *Ejercicio 3. Veinte millones de sumas con 8 hilos, en la máquina de referencia con Windows. Con double, atomic no mejora a critical: es incluso un poco más lento. Con long long es tres veces más rápido. Y reduction queda por debajo incluso del bucle secuencial.*

El ensamblador dice por qué. Con un entero, la suma atómica es **una instrucción** del procesador, `lock add`. Con un `double` esa instrucción no existe, y el compilador genera un bucle: leer el valor, sumar, y escribirlo con `lock cmpxchg` solo si nadie lo ha cambiado entretanto; si otro hilo se ha adelantado, vuelta a empezar. Con 8 hilos peleando por la misma variable se reintenta tanto que cuesta lo que un cerrojo. Es lo que cuenta el final del apartado 3.3.8.

¿Y por qué salen dos de cada? Porque `reduction` también usa una actualización atómica: la que suma la copia de cada hilo al resultado al terminar. En el ensamblador de la máquina de referencia, una `lock cmpxchg` está en la región de `d_atomic` y la otra en la de `d_reduction`, y lo mismo con `lock add` en las de `long long`. La diferencia no es la instrucción: es que en `atomic` se ejecuta veinte millones de veces y en `reduction`, una vez por hilo.

En Linux, sobre la misma máquina, `critical` es mucho más caro (1,7195 s con `double`), así que ahí `atomic` con `double` sí gana, 2,43 veces; pero con `long long` gana 7,38. El tipo sigue mandando. Y la fila más incómoda es la primera: el bucle secuencial tarda 0,0118 s. Con `critical` y 8 hilos, el programa es **más de 60 veces más lento que no paralelizar nada**.

> **PISTA** Tus cuatro funciones son copias de las de `double` cambiando el tipo del acumulador y sumando `v[i]`. Para contar en el ensamblador basta con buscar el texto: en bash, `grep -c "lock cmpxchg" p3_ej3.s`; en PowerShell, `(Select-String "lock cmpxchg" p3_ej3.s).Count`. Comprueba al final que la última línea de la salida dice «lo mismo»: si un mecanismo fuera más rápido pero diera otro resultado, la comparación no valdría.

Qué va en el PDF

- Tus cuatro funciones con `long long`, pegadas como texto.
- La salida completa y una gráfica de barras con escala logarítmica de los ocho tiempos.
- Cuántas `lock cmpxchg` y `lock add` encontraste y en qué región paralela está cada una: en el ensamblador, las regiones salen como funciones con el nombre de la tuya seguido de `._omp_fn`.
- Dos o tres conclusiones justificadas: por qué `atomic` se comporta distinto según el tipo, qué demuestra el experimento de control y qué significa que `critical` sea más lento que el secuencial.

Trabajo autónomo obligatorio: ejercicios 4 y 5 (4 puntos)

Se hacen fuera del aula, con el mismo entorno. Como en el núcleo, los programas se dan casi completos y en cada uno falta la pieza que enseña el ejercicio; en el 5, esa pieza es la función recursiva entera. La estimación es de unas tres horas entre los dos, y no necesitan nada que no esté en los apartados 3.3 y 3.4.

Ejercicio 4 (2,0 puntos) — Compartir sin querer: el temporal y la falsa compartición

Enunciado. Dos programas cortos con la misma raíz: los hilos comparten algo que no deberían. En la **parte A** comparten una variable, y el fallo es de **corrección**: el resultado sale mal o, peor, sale bien por casualidad. En la **parte B** no comparten ningún dato, solo la línea de caché, y el fallo es de **rendimiento**: todo sale bien y va varias veces más lento. Cada parte vale un punto.

Parte A (1,0 punto) — El temporal que creías privado

Enunciado. `p3_ej4a.c` tiene el error número uno de OpenMP (apartado 3.3.3) en cuatro versiones, y compara cada una con la secuencial posición a posición, veinte ejecuciones por versión. Se dan escritas las dos primeras, que son el mismo error:

- **A**: una variable auxiliar declarada **fuera** del bucle y usada dentro. Es compartida por defecto.
- **A2**: la misma, pero declarada `volatile`.

Tu parte: escribir las otras dos, **B** con `private(t)` y **C** con la variable declarada dentro del bucle y la región con `default(none)`. Después, compila el mismo fichero con `-O2` y con `-O0` y ejecuta los dos. Y por último, añade `default(none) shared(x, y)` a la versión A, sin nombrar `t`, y compila.

```c
/* p3_ej4a.c - el temporal que creias privado (apartado 3.3.3).
   Cada version se ejecuta EJEC veces y se compara con la secuencial posicion a
   posicion. Compilalo con -O2 y con -O0 y compara las dos salidas. */
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <omp.h>

#define N    4000000
#define EJEC 20

static double *x, *y, *ref;

/* Posiciones en las que y no coincide con la referencia secuencial. */
static long long difiere(void)
{
    long long d = 0;
    for (int i = 0; i < N; i++)
        if (y[i] != ref[i]) d++;
    return d;
}

static void informa(const char *nombre, int malas, long long peor)
{
    printf("%-38s %5d/%d %14lld\n", nombre, malas, EJEC, peor);
}

/* A: la variable auxiliar se declara fuera de la region */
static void version_a(void)
{
    int malas = 0; long long peor = 0;
```

```
    for (int e = 0; e < EJEC; e++) {
❶       double t;                              /* fuera de la region: COMPARTIDA */
        #pragma omp parallel for shared(t)
        for (int i = 0; i < N; i++) {
            t = sqrt(x[i]) + 1.0;
            y[i] = t * t;
        }
        long long d = difiere();
        if (d) { malas++; if (d > peor) peor = d; }
    }
    informa("A  temporal compartida", malas, peor);
}

/* A2: el mismo error, con la variable obligada a vivir en memoria */
static void version_a2(void)
{
    int malas = 0; long long peor = 0;
    for (int e = 0; e < EJEC; e++) {
❷       volatile double t;                     /* volatile: nunca en un registro */
        #pragma omp parallel for shared(t)
        for (int i = 0; i < N; i++) {
            t = sqrt(x[i]) + 1.0;
            y[i] = t * t;
        }
        long long d = difiere();
        if (d) { malas++; if (d > peor) peor = d; }
    }
    informa("A2 la misma, con t volatile", malas, peor);
}

/* B: la misma variable de fuera, con private(t).
   EJERCICIO 4: escribela partiendo de version_a. */
❸ static void version_b(void)
{
    /* tu codigo */
}

/* C: la variable declarada dentro del bucle, y la region con default(none).
   EJERCICIO 4: escribela tambien. */
static void version_c(void)
{
    /* tu codigo */
}

int main(void)
{
    x = malloc(N * sizeof *x);
    y = malloc(N * sizeof *y);
    ref = malloc(N * sizeof *ref);
    if (!x || !y || !ref) { fprintf(stderr, "sin memoria\n"); return 1; }
    for (int i = 0; i < N; i++) {
        x[i] = 1.0 + (double)(i % 1000);
        double t = sqrt(x[i]) + 1.0;           /* la version secuencial */
        ref[i] = t * t;
    }
    printf("%d hilos, %d posiciones, %d ejecuciones por version\n",
           omp_get_max_threads(), N, EJEC);
    printf("version                                ejecuciones mal   posiciones mal\n");
```

```
    version_a();
    version_a2();
    version_b();
    version_c();
    free(x);
    free(y);
    free(ref);
    return 0;
}
```

❶ Aquí está el fallo: `t` se declara fuera de la región, así que los 24 hilos comparten **una sola** variable. Cada uno escribe la suya y lee la que haya en ese momento, que puede ser la de otro. El `shared(t)` está escrito para que se vea; sin él pasaría lo mismo. ❷ El mismo error, con la variable obligada a vivir en memoria. ❸ Aquí empiezan tus dos versiones.

Resultado en la máquina de referencia con Windows, primero con `-O2` y después con `-O0`:

```
$ gcc -O2 -fopenmp -Wall -Wextra -o p3_ej4a p3_ej4a.c -lm
24 hilos, 4000000 posiciones, 20 ejecuciones por version
version                                ejecuciones mal   posiciones mal
A  temporal compartida                     0/20              0
A2 la misma, con t volatile               20/20         229875
B  private(t)                              0/20              0
C  declarada dentro + default(none)        0/20              0

$ gcc -O0 -fopenmp -Wall -Wextra -o p3_ej4a p3_ej4a.c -lm
24 hilos, 4000000 posiciones, 20 ejecuciones por version
version                                ejecuciones mal   posiciones mal
A  temporal compartida                    20/20         405460
A2 la misma, con t volatile               20/20         447224
B  private(t)                              0/20              0
C  declarada dentro + default(none)        0/20              0
```

La versión A, que es incorrecta, **no falla ni una sola vez con -O2** y falla las veinte con `-O0`. El fichero es el mismo y el error es el mismo: solo ha cambiado una opción de compilación. En Linux sale igual.

La explicación: con `-O2` el compilador ve que `t` se escribe e inmediatamente se lee, y la deja en un registro. Un registro es privado de cada hilo por construcción, así que el programa se comporta *como si* tuviera `private(t)`. No lo tiene. Basta con que el bucle se complique, o con cambiar de compilador, para que la variable vuelva a memoria y el fallo aparezca: es lo que hace `volatile` en A2. **Que un programa concurrente dé el resultado bueno no demuestra que sea correcto.** Y por eso existe `default(none)`, que convierte este error silencioso en uno de compilación:

```
$ gcc -O2 -fopenmp -c p3_ej4a.c
p3_ej4a.c:36:15: error: 't' not specified in enclosing 'parallel'
```

> **PISTA** Las dos versiones que escribes son copias de `version_a` con un cambio cada una. En C, fíjate en qué variables usa el bucle: con `default(none)` hay que nombrar también `x` e `y`, porque están declaradas fuera, y no hace falta nombrar `N`, que es una constante del preprocesador. Si con `-O0` la versión A tampoco te falla, comprueba que de verdad estás usando varios hilos.

> **Qué va en el PDF, parte A**
>
> - Tus versiones B y C, pegadas como texto.
> - Las dos salidas, con la orden de compilación exacta encima de cada una, y el mensaje de error de `default(none)` sobre la versión A.
> - Dos o tres conclusiones justificadas: por qué `-O2` esconde el fallo, qué significa eso para probar programas concurrentes y qué te habría ahorrado `default(none)`.

Parte B (1,0 punto) — Falsa compartición y el escalón de la línea de caché

Enunciado. En `p3_ej4b.c` cada hilo incrementa **su propia casilla** de un array, sin compartir ningún dato con nadie, y aun así el programa va mucho más lento de lo que debería (apartado 3.3.12). El programa mide las casillas pegadas y separadas, con 2 a 24 hilos, y después barre la separación de 1 a 16 casillas con 8 hilos. Además pregunta al procesador, con la instrucción `CPUID`, cuánto mide su línea de caché.

- **Tu parte**: la función `con_reduction`, el mismo recuento sin array ninguno. Ya sabes escribirla: es la `reduction` que mediste en el ejercicio 3.
- Comprueba que las tres versiones dan el total exacto: **aquí no hay ninguna carrera**, y eso es lo que hace tan difícil encontrar este problema.

```c
/* p3_ej4b.c - falsa comparticion y el escalon de la linea de cache (apartado 3.3.12).
   Cada hilo incrementa SU casilla; lo unico que cambia es la distancia entre ellas. */
#include <stdio.h>
#include <cpuid.h>
#include <omp.h>
#include "crono.h"

#define VUELTAS 200000000    /* incrementos en total, repartidos entre los hilos */
#define MAXH    64
#define SEPMAX  16
#define REP     5

❶ static volatile double sitio[MAXH * SEPMAX];

/* Tamano de la linea de cache que declara el procesador: instruccion CPUID. */
static int linea_cache(void)
{
    unsigned a, b, c, d;
    if (!__get_cpuid(1, &a, &b, &c, &d)) return 0;
❷   return (int)((b >> 8) & 0xff) * 8;
}

/* Cada hilo suma uno VUELTAS/p veces en sitio[yo * sep]. */
static double mide(int p, int sep, double *total)
{
    for (int i = 0; i < MAXH * SEPMAX; i++) sitio[i] = 0.0;
    double t0 = reloj();
    #pragma omp parallel num_threads(p)
    {
        int yo = omp_get_thread_num();
        for (int k = 0; k < VUELTAS / p; k++)
❸           sitio[yo * sep] += 1.0;
    }
    double dt = reloj() - t0;
    double s = 0.0;
```

```
    for (int i = 0; i < MAXH * SEPMAX; i++) s += sitio[i];
    *total = s;
    return dt;
}

/* El mismo recuento sin array ninguno: cada hilo en su copia, y reduction.
   EJERCICIO 4: escribela. Cada hilo suma 1.0 VUELTAS/p veces; deja en *total
   la suma y devuelve el tiempo. */
❹ static double con_reduction(int p, double *total)
{
    /* tu codigo */
}

/* Mejor de REP de mide(p, sep); sep = 0 significa con_reduction. */
static double mejor(int p, int sep, double *total)
{
    double m = 1e30;
    for (int r = 0; r < REP; r++) {
        double t = sep ? mide(p, sep, total) : con_reduction(p, total);
        if (t < m) m = t;
    }
    return m;
}

int main(void)
{
    printf("linea de cache segun CPUID: %d bytes; un double ocupa %d\n",
           linea_cache(), (int)sizeof(double));
    int hilos[] = {2, 4, 8, 24};
    double tp, ts, tr, sp, ss, sr;
    printf(" hilos  pegadas(s)  separadas(s)  factor  reduction(s)  sumas\n");
    for (int k = 0; k < 4; k++) {
        int p = hilos[k];
        tp = mejor(p, 1, &sp);
        ts = mejor(p, SEPMAX, &ss);
        tr = mejor(p, 0, &sr);
        double esperado = (double)(VUELTAS / p) * p;
        printf("%6d %11.4f %13.4f %6.1fx %13.4f  %s\n", p, tp, ts, tp / ts, tr,
               (sp == esperado && ss == esperado && sr == esperado) ? "bien" : "MAL");
    }
    printf("barrido de la separacion con 8 hilos\n separacion  bytes  tiempo(s)\n");
    for (int sep = 1; sep <= SEPMAX; sep++)
        printf("%11d %6d %10.4f\n", sep, sep * (int)sizeof(double), mejor(8, sep, &sp));
    return 0;
}
```

❶ `volatile`, para que las casillas vivan en memoria (trampa 4). ❷ `CPUID` con la hoja 1 devuelve en el registro `EBX` el tamaño de línea en bloques de 8 bytes. ❸ Todo el experimento: `sep` es la distancia entre las casillas de dos hilos consecutivos. ❹ Aquí va tu función.

Resultado en la máquina de referencia con Windows:

```
linea de cache segun CPUID: 64 bytes; un double ocupa 8
 hilos  pegadas(s)  separadas(s)  factor  reduction(s)  sumas
     2      1.0978        0.3628    3.0x        0.0907  bien
     4      1.5740        0.1822    8.6x        0.0457  bien
     8      0.8317        0.0918    9.1x        0.0229  bien
    24      0.5443        0.0466   11.7x        0.0117  bien
barrido de la separacion con 8 hilos
 separacion  bytes  tiempo(s)
          1      8     0.8325
          2     16     0.8210
          3     24     0.5945
          4     32     0.4200
          5     40     0.4187
          6     48     0.4080
          7     56     0.3566
          8     64     0.0913
          9     72     0.0911
         10     80     0.0908
         11     88     0.0911
         12     96     0.0912
         13    104     0.0913
         14    112     0.0913
         15    120     0.0913
         16    128     0.0908
```

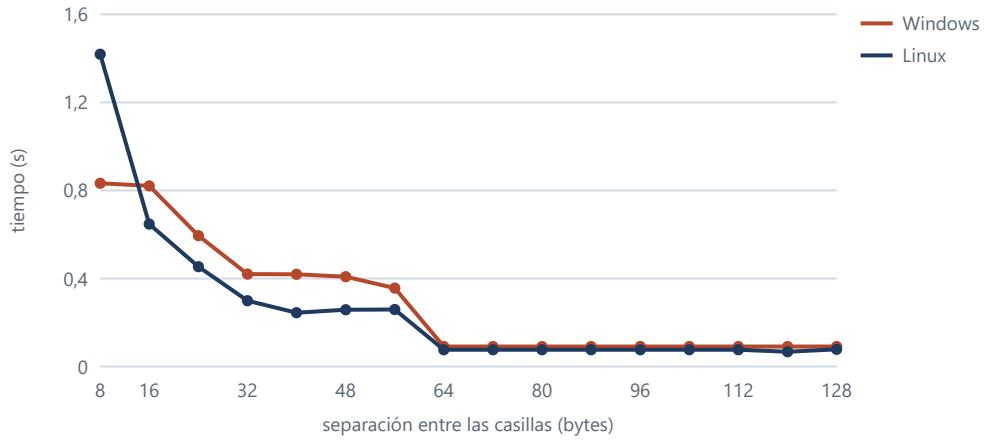


Figura P3.3 · *Ejercicio 4. El mismo programa con 8 hilos, cambiando solo la distancia entre las casillas. En los dos sistemas el tiempo cae de golpe entre 56 y 64 bytes, justo donde dos casillas dejan de caber en la misma línea de caché de 64 bytes que declara el procesador, y después ya no baja más.*

El barrido es lo mejor del ejercicio. Con 56 bytes de separación el programa tarda 0,3566 s; con 64, 0,0913 s; y a partir de ahí ya no mejora, por mucho que separes. El escalón no está en un sitio cualquiera: está **exactamente** donde dos casillas dejan de caber en la misma línea de 64 bytes, el tamaño que acaba de declarar el propio procesador. Eso es una prueba, no una ilustración: si la caída estuviera en 32 o en 128 bytes, la explicación por líneas de caché sería falsa. En Linux, el escalón cae en el mismo sitio.

Y la columna de `reduction` cierra el asunto: es cuatro veces más rápida que la versión separada, no necesita saber cuánto mide tu línea de caché y es una cláusula. La moraleja no es «aprende a separar casillas», es **«no montes un array de acumuladores si reduction hace el trabajo»**.

PISTA Si tu `CPUID` dice otro tamaño de línea, el escalón tiene que moverse con él: compruébalo en tu barrido. Y si tu `con_reduction` sale más lenta que la versión separada, mira si has declarado el acumulador `volatile`: la copia privada de cada hilo no lo necesita, y sin él se queda en un registro.

Qué va en el PDF, parte B

- Tu función `con_reduction`, pegada como texto.
- La salida completa, con el tamaño de línea que dio `CPUID`, y la gráfica del barrido de separación.
- Dos o tres conclusiones justificadas: por qué hay falsa compartición sin datos compartidos, por qué el escalón cae donde cae y cuándo usarías cada solución.

Ejercicio 5 (2,0 puntos) — Tareas: cuánto trabajo merece una tarea

Enunciado. `p3_ej5.c` ordena diez millones de enteros con mergesort, primero en serie y después con tareas, que es la forma natural de paralelizar un divide y vencerás (apartados 3.4.3 y 3.4.4). La versión con tareas lleva un **umbral de corte**: un trozo con menos elementos que el umbral ya no se parte en tareas y se ordena en serie (apartado 3.4.5). El programa barre el umbral de diez a un millón, cuenta sin ejecutarlas cuántas tareas crea cada valor y comprueba que el vector sale idéntico al de la versión secuencial.

- **Antes de medir**, predice por escrito con la regla práctica del apartado 3.4.5 —entre 10 y 100 tareas por hilo— qué umbral va a ser el mejor en tu máquina, y cuál va a ir más lento que no paralelizar.
- **Tu parte**: la función `ms_par`. Si el trozo tiene menos de `umbral` elementos, lo ordena en serie con `ms_sec`; si no, ordena la mitad izquierda y la derecha como dos tareas, espera a las dos y las mezcla.

```c
/* p3_ej5.c - mergesort con tareas y el umbral de corte (apartados 3.4.3 a 3.4.5).
   Ordena el mismo vector en serie y con tareas, barriendo el umbral por debajo
   del cual ya no se crean tareas. */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <omp.h>
#include "crono.h"

#define N   10000000
#define REP 3

static int *orig, *v, *tmp, *ref;

/* Mezcla v[izq..med) y v[med..der), que ya estan ordenados, usando tmp. */
❶ static void mezcla(int izq, int med, int der)
{
    int i = izq, j = med, k = izq;
    while (i < med && j < der) tmp[k++] = (v[i] <= v[j]) ? v[i++] : v[j++];
    while (i < med) tmp[k++] = v[i++];
    while (j < der) tmp[k++] = v[j++];
    memcpy(v + izq, tmp + izq, (size_t)(der - izq) * sizeof *v);
}

/* Ordena v[izq..der) en serie. */
static void ms_sec(int izq, int der)
{
    if (der - izq < 2) return;
    int med = izq + (der - izq) / 2;
```

```
    ms_sec(izq, med);
    ms_sec(med, der);
    mezcla(izq, med, der);
}

/* Ordena v[izq..der) con tareas; los trozos de menos de umbral elementos, en
   serie. EJERCICIO 5: escribela. */
❷ static void ms_par(int izq, int der, int umbral)
{
    /* tu codigo */
}

/* Cuantas tareas crea ms_par con ese umbral sobre n elementos, sin ejecutarla. */
❸ static long long tareas(int n, int umbral)
{
    if (n < umbral) return 0;
    return 2 + tareas(n / 2, umbral) + tareas(n - n / 2, umbral);
}

int main(void)
{
    orig = malloc(N * sizeof *orig);
    v = malloc(N * sizeof *v);
    tmp = malloc(N * sizeof *tmp);
    ref = malloc(N * sizeof *ref);
    if (!orig || !v || !tmp || !ref) { fprintf(stderr, "sin memoria\n"); return 1; }
    unsigned x = 12345u;                        /* el mismo desorden siempre */
    for (int i = 0; i < N; i++) { x = x * 1103515245u + 12345u; orig[i] = (int)(x >> 1); }

    int hilos = omp_get_max_threads();
    printf("%d hilos, %d enteros, mejor de %d\n", hilos, N, REP);
    double ts = 1e30;
    for (int r = 0; r < REP; r++) {
        memcpy(v, orig, N * sizeof *v);
        double t0 = reloj();
        ms_sec(0, N);
        double t = reloj() - t0;
        if (t < ts) ts = t;
    }
    memcpy(ref, v, N * sizeof *v);              /* la respuesta correcta */
    int ordenado = 1;
    for (int i = 1; i < N; i++) if (ref[i - 1] > ref[i]) ordenado = 0;
    printf("secuencial %.4f s, %s\n", ts, ordenado ? "ordenado" : "MAL ORDENADO");

    #pragma omp parallel
❹   { }                                         /* crea el equipo fuera del cronometro */

    int umbrales[] = {10, 100, 1000, 10000, 100000, 1000000};
    printf("  umbral     tareas  por hilo  tiempo(s)  speedup  resultado\n");
    for (int k = 0; k < 6; k++) {
        double tp = 1e30;
        for (int r = 0; r < REP; r++) {
            memcpy(v, orig, N * sizeof *v);
            double t0 = reloj();
            #pragma omp parallel
❺           #pragma omp single
            ms_par(0, N, umbrales[k]);
            double t = reloj() - t0;
```

```
            if (t < tp) tp = t;
        }
        long long nt = tareas(N, umbrales[k]);
        printf("%8d %10lld %9.1f %10.4f %7.2fx  %s\n", umbrales[k], nt, (double)nt / hilos,
               tp, ts / tp, memcmp(v, ref, N * sizeof *v) == 0 ? "igual" : "DISTINTO");
    }
    free(orig);
    free(v);
    free(tmp);
    free(ref);
    return 0;
}
```

❶ La mezcla de dos mitades ya ordenadas, con el vector auxiliar `tmp`. Es secuencial, y la última recorre los diez millones de elementos. ❷ Aquí va tu función. ❸ Cuenta las tareas que crearía `ms_par` recorriendo el mismo árbol sin ordenar nada: así el número es exacto y no se paga dentro del cronómetro. ❹ La primera región paralela crea el equipo de hilos, y se abre aquí, fuera del cronómetro, como en el apartado 3.4.5. ❺ El envoltorio de siempre: `parallel` abre el equipo y `single` hace que un solo hilo empiece la recursión; los demás esperan en la barrera sacando tareas de la cola (apartado 3.4.3).

Resultado en la máquina de referencia con Windows y 24 hilos, con tu función ya escrita:

```
24 hilos, 10000000 enteros, mejor de 3
secuencial 0.7802 s, ordenado
  umbral     tareas  por hilo  tiempo(s)  speedup  resultado
      10    3222782  134282.6     3.4870    0.22x  igual
     100     262142   10922.6     0.4138    1.89x  igual
    1000      32766    1365.3     0.1335    5.84x  igual
   10000       2046      85.3     0.1292    6.04x  igual
  100000        254      10.6     0.1290    6.05x  igual
 1000000         30       1.3     0.1322    5.90x  igual
```

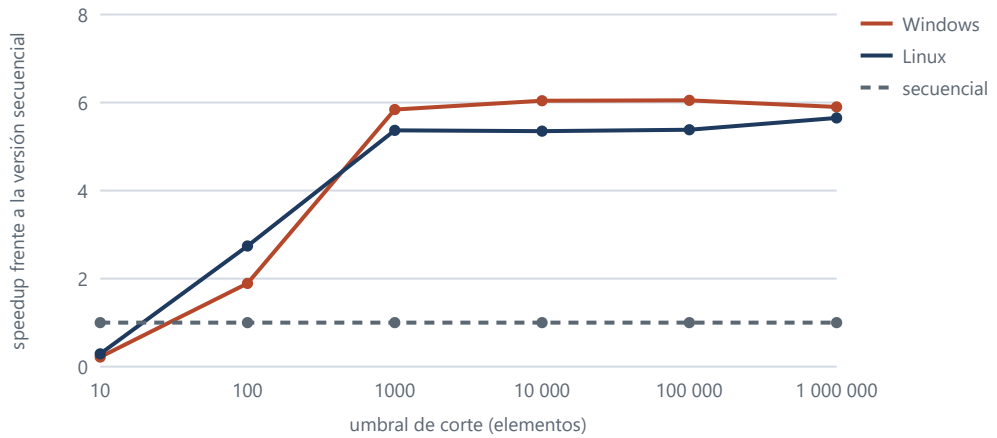


Figura P3.4 · *Ejercicio 5. El mergesort con tareas frente al umbral de corte, con 24 hilos y diez millones de enteros. Por debajo de la línea gris, paralelizar sale más caro que ordenar en serie. De mil a un millón de elementos de umbral la curva es casi plana: lo que hay que acertar es la década, no el número.*

Con umbral 10 el programa crea 3 222 782 tareas, 134 283 por hilo, y tarda 3,4870 s: **4,5 veces más lento que ordenar en serie**. Con 100 apenas gana. A partir de 1000 el tiempo se aplana: entre mil y un millón de elementos de umbral, un factor de mil, el speedup solo se mueve entre 5,84 y 6,05. La regla de la teoría acierta la zona: con 10 000 y 100 000 salen 85,3 y 10,6 tareas por hilo. En Linux la forma es la misma: 0,29 con umbral 10 y una meseta entre 5,35 y 5,65.

Queda la pregunta mejor del ejercicio: con 24 hilos y trabajo de sobra, **¿por qué ningún umbral pasa de unas seis veces?** La respuesta no está en las tareas, está en el final del algoritmo: piensa qué hace la última mezcla y cuántos hilos pueden trabajar en ella.

PISTA En una tarea, las variables locales de `ms_par` — `izq`, `med`, `der`, `umbral` — pasan por copia, que es justo lo que quieres: son `firstprivate` por defecto (apartado 3.4.3). `v` y `tmp` son globales y compartidas, y no hay carrera porque cada tarea escribe en su trozo sin solaparse con ninguna otra. Y el `taskwait` no es opcional: sin él, cada mezcla empieza antes de que sus dos mitades estén ordenadas. En la máquina de referencia, el programa sin esa línea llega a ir 14,73 veces más rápido que el secuencial y las seis filas dicen DISTINTO. Rápido y mal no es un speedup.

Qué va en el PDF

- Tu predicción, escrita antes de medir, y tu función `ms_par`, pegada como texto.
- La tabla completa y una gráfica del speedup frente al umbral, con el eje del umbral en escala logarítmica.
- Dos o tres conclusiones justificadas: si acertó tu predicción y por qué, por qué el umbral más pequeño va más lento que en serie y por qué ningún umbral pasa de unas seis veces con 24 hilos.

Entrega y evaluación

Se entrega **un único PDF por equipo**, llamado `CAR_P3_Apellidos1_Apellidos2_Apellidos3_Apellidos4.pdf`, con los apellidos de las cuatro personas del grupo separados por guion bajo. **Solo ese PDF**: no se entregan ficheros de código sueltos, ni carpetas, ni archivos comprimidos, ni enlaces a repositorios. El núcleo se trabaja en clase, y el PDF completo —núcleo y trabajo autónomo— se entrega en la fecha que indique el campus virtual.

No hace falta un informe largo. Dentro del PDF va esto:

- Una **portada** con los nombres completos del equipo, el modelo de procesador con sus núcleos físicos y lógicos, el sistema operativo y el compilador con su versión.
- Para cada ejercicio, **el código pegado como texto**. No valen fotos de la pantalla del código.
- Para cada ejercicio, una **tabla breve** con tus medidas o la captura de la salida, diciendo cuántas repeticiones hay detrás.
- Las **gráficas** de los ejercicios 2 a 5. Sirve cualquier herramienta: una hoja de cálculo basta.
- Para cada ejercicio, **dos o tres conclusiones justificadas** con tus números: las que pide su bloque «Qué va en el PDF».
- Al final, una **declaración del trabajo**: qué ha hecho cada integrante y si habéis usado IA, para qué y qué habéis comprobado vosotros. La IA puede ayudar a escribir o a revisar código; no sustituye a medir ni a entender lo medido, y cada integrante tiene que poder defender cualquier número del PDF.

Cada ejercicio pesa lo que dice su título, y los cinco suman 10:

EJERCICIO	QUÉ SE MIRA SOBRE TODO	PESO
1 · Cuáles se pueden y cuáles no	La clasificación previa, el bucle 3 arreglado y por qué falla en tan pocas posiciones	15 %
2 · Planificaciones	La función `reparto` y por qué `static` es la más lenta aunque reparta iteraciones por igual	20 %
3 · critical, atomic y reduction	El experimento de control con `long long` y la explicación con el ensamblador	25 %
4 · Compartir sin querer	Las versiones B y C y por qué `-O2` esconde el fallo; el escalón puesto frente al tamaño de línea de `CPUID`	20 %
5 · Tareas y umbral de corte	La predicción escrita antes de medir, el barrido y por qué el speedup se aplana	20 %
Total		**100 %**

Y dentro de cada ejercicio, la nota se reparte así:

CRITERIO	QUÉ SIGNIFICA	PESO
Medida y corrección comprobadas	Reloj adecuado, repeticiones suficientes, resultados comprobados contra el secuencial y herramientas comprobadas antes de usarlas	30 %
Conclusiones	Se entiende **por qué** salen esos números. Una tabla sin conclusiones no puntúa	35 %
Presentación de los datos	Tablas legibles, gráficas con ejes rotulados y unidades en su sitio	20 %
Código	Compila sin avisos con `-Wall -Wextra`; en el código que escribes tú, además, nombres con sentido y sangrado uniforme	15 %
Total		**100 %**

Nota de la práctica = ejercicios 1 a 5, hasta 10.

> **CÓMO SE CORRIGEN LAS CONCLUSIONES** No se busca que tus números coincidan con los de este guion. Se busca que sepas defender lo que has medido: qué mediste, con qué reloj, cuántas veces, qué esperabas y por qué salió otra cosa. Un resultado raro bien explicado puntúa más que un resultado limpio sin analizar.

ANTES DE CERRAR EL PORTÁTIL

Cinco minutos que evitan la mitad de los suspensos de esta práctica.

¿Da 0 fallos tu bucle 3 arreglado? ¿Dice «identico al secuencial» el ejercicio 2, «lo mismo» el 3, «bien» la parte B del 4 e «igual» el 5? ¿Has compilado la parte A del ejercicio 4 con `-O2` y con `-O0`? ¿Escribiste la predicción del ejercicio 5 antes de medir? ¿Cada tabla dice cuántas repeticiones hay detrás y en qué sistema se midió?